CMPE 492
Nounce: Language Platform for Phonetic Analysis & Pronunciation Assessment

Ümit Can Evleksiz
Ömer Faruk Bayram

Advisor:
Prof. Lale Akarun
Prof. Murat Saraçlar

Contents

Chapter 1

INTRODUCTION

1.1 Broad Impact

1.1.1 Learning Impact

Nounce is a web-based pronunciation assessment application that provides learners with practical how-to knowledge for speaking standard English, focusing on pronunciation accuracy and clarity. By introducing users to the International Phonetic Alphabet (IPA) and phonetic concepts, the system builds domain familiarity, enabling learners to discuss and understand sound units systematically. The platform incorporates gamification elements and provides fast, friendly feedback to create a safe learning environment where users can practice without fear of judgment.

It is important to note that the objective is not to replace human-to-human interactions in language learning, but rather to provide accessible tools for individuals who may lack access to human tutors, native speaker friends, or formal language education resources. The system serves as a supplementary tool that empowers self-directed learning while acknowledging the irreplaceable value of human interaction in language acquisition.

1.1.2 Academic Impact

This project contributes to the academic community by demonstrating the practical application of fine-tuning deep learning models for specialized phonetic transcription tasks. The work combines core signal processing techniques with modern machine learning approaches, integrating multiple moving parts including audio preprocessing, phonetic transcription, forced alignment, and pronunciation assessment into a cohesive full-stack application. The system showcases how research-grade models can be adapted and deployed in real-world educational contexts,

providing a reference implementation for similar pronunciation assessment systems.

1.2 Ethical Considerations

1.2.1 User Speech Data

The system handles sensitive user speech data, which raises important privacy and usage concerns. Audio recordings are used for inference during pronunciation assessment, but users must explicitly opt-in for any potential use of their data in model fine-tuning, voice cloning, or correct pronunciation generation features. Clear consent mechanisms and transparent data usage policies are essential to protect user privacy and maintain trust.

1.2.2 Prescriptive Approach in Linguistics

The application focuses on US English pronunciation, which may appear prescriptive. However, it is crucial to emphasize that all dialects, accents, and language variants are linguistically valid. US English is chosen not to minimize or devalue other accents, but because it is widely adopted and in high demand globally, particularly in professional and academic contexts. The system aims to improve comprehensibility and communication effectiveness rather than eliminate linguistic diversity. Users are encouraged to understand that accent reduction is optional and that the primary goal is clear communication, not accent elimination.

1.2.3 Monetization and Accessibility

Commercialization of educational technology raises concerns about potentially gatekeeping education from economically disadvantaged populations. While the system may include premium features or subscription models, careful consideration must be given to ensuring that core pronunciation assessment capabilities remain accessible. Free tiers, educational discounts, and open-source components can help mitigate barriers to access, ensuring that pronunciation learning tools do not become exclusive to those who can afford them.

Chapter 2

PROJECT DEFINITION AND PLANNING

2.1 Project Definition

This project develops Nounce, a web-based application to assist English language learners in improving pronunciation through automated speech analysis and personalized feedback. The system enables users to record speech, receive detailed pronunciation assessments, and gain insights into English phonology.

Core functionality includes: (i) capturing and processing user speech recordings, (ii) analyzing pronunciation accuracy at the phonetic level using machine learning models, (iii) comparing actual pronunciation against target acoustic patterns, and (iv) providing educational guidance on English phonology.

The system focuses on English with US accent (potentially including UK), with initial development targeting Turkish-native English speakers. Future milestones will leverage users' native language to better detect phonetic units. The application is designed to be accessible and pedagogically sound, emphasizing comprehensibility over accent elimination.

2.1.1 Relationship to the CMPE491 Prototype

Nounce builds directly on a prototype developed for CMPE491. That first version established the end-to-end recording-to-feedback concept, but several of its components proved unreliable or pedagogically thin. The present work (CMPE492) reworks the system around two themes—*reliability* and *deviation-awareness*—and the improvements over the prototype are detailed in the relevant sections rather than enumerated here. In brief: forced alignment moves from the Montreal Forced Aligner to POWSM's own CTC alignment, giving a single consistent phone in-

ventory and real frame-level timestamps (Chapter 6); synthesized text-to-speech reference audio is retired in favour of human native recordings in General American and Received Pronunciation; an audio-quality stage now detects no-speech, noisy, and wrong-sentence input and abstains rather than returning a misleading score; the phone-level difference and the learner-facing feedback are computed over articulatory feature vectors instead of exact-match string comparison, so they reflect *how far* a produced phone is from the target; and the interface adds per-phone playback with synchronized highlighting and goodness-of-pronunciation colouring. Most significantly, where the prototype shipped without any formal evaluation, this version introduces dedicated validation sets, a cross-dataset fine-tuning study, and a human-judgment correlation analysis (Section 8.2). The through-line is a shift from a transcribe-and-compare prototype to an evaluated assessment system that detects and explains pronunciation deviations.

2.2 Project Planning

2.2.1 Project Time and Resource Estimation

The project is developed as a two-person team effort with the following resource allocation:

Time Commitment:

- Development time: 15 hours per week per team member
- Project duration: Two academic semesters (CMPE 491 + CMPE 492)
- Total estimated effort: 360 hours per member

Infrastructure Costs:

- Full-stack server hosting: $5 per month
- Serverless PostgreSQL database: $5 per month
- Serverless GPU for ML job queue: $20-30 per month (moderate user activity)
- Total monthly infrastructure cost: $30-40 per month

Additional Resources:

- Development tools and frameworks (open-source)
- ML model training and evaluation datasets
- Audio processing libraries and APIs

2.2.2 Success Criteria

The project will be considered successful upon achieving the following objectives:

Technical Requirements:

- Full-stack secure web application with authentication and authorization
- System uptime of 99% or higher
- Acoustic ML model producing accurate phonetic-level transcriptions independent of prosodic variations
- Difference algorithm comparing actual pronunciation against target patterns with actionable feedback
- Scalable architecture supporting concurrent sessions and asynchronous audio processing

User Experience Requirements:

- Industry-grade UI/UX following modern web standards
- Intuitive workflow for recording, uploading, and reviewing feedback
- Clear presentation of pronunciation analysis results
- Responsive design supporting multiple devices
- Accessible interface adhering to web accessibility guidelines

Functional Requirements:

- Accurate real-time or near-real-time pronunciation assessment
- Educational content integration for English phonology
- Progress tracking and historical analysis
- Support for multiple audio formats and quality levels

2.2.3 Risk Analysis

Several risks have been identified that may impact project success:

Technical Risks:

- **ML Model Performance:** The machine learning model may not achieve desired accuracy, particularly with low-quality audio, background noise, or non-standard accents. Mitigation includes extensive training on diverse datasets, robust preprocessing, and fallback mechanisms for low-confidence predictions.
- **Audio Quality Variability:** Users may submit recordings with varying quality, device capabilities, and environmental conditions. The system must handle this through adaptive preprocessing and quality assessment.
- **Computational Resource Constraints:** Processing requirements may strain serverless GPU resources during peak usage. Load balancing, queue management, and resource scaling will address this.
- **Integration Complexity:** Coordinating multiple system components may introduce integration challenges. Comprehensive testing and modular architecture will mitigate these risks.

Pedagogical and Ethical Risks:

- **Language and Accent Limitations:** The application focuses on English with US accent (potentially UK). Other languages are not supported. Initial development targets Turkish-native English speakers, with future milestones leveraging native language for better phonetic detection. This limitation must be clearly communicated to users to avoid misconceptions about supported languages and accents.
- **Overly Prescriptive Approach:** The system may promote misconceptions about a single "correct" pronunciation, marginalizing linguistic variation. This is addressed by emphasizing comprehensibility over accent matching, acknowledging linguistic diversity, and transparently communicating system limitations.
- **User Expectations vs. Reality:** Users may have unrealistic expectations about pronunciation transformation. Clear communication about the tool's purpose as a learning aid is essential.
- **Data Privacy and Security:** Handling sensitive user audio data requires robust security measures and clear privacy policies. Users must explicitly

opt-in for any use of their data in model fine-tuning, voice cloning, or pronunciation generation. Compliance with data protection regulations will be prioritized.

- **Accessibility and Monetization:** Commercial features may create barriers for economically disadvantaged users. Free tiers and educational discounts should be considered to ensure accessibility.

Project Management Risks:

- **Coordination Overhead:** With two team members working on different components, ensuring consistent integration and avoiding conflicting changes requires regular communication and code review.
- **Scope Creep:** The project may expand beyond initial scope. Potential scope creep includes voice cloning and generating correct pronunciation using the user's voice, which are explicitly out of scope. A clear feature prioritization framework and milestone-based development approach will maintain focus.

2.2.4 Team Work

This project is developed as a two-person team effort with complementary responsibilities:

- **Ümit Can Evleksiz:** Full-stack web application development, ML pipeline integration (assessment and IPA generation endpoints), audio preprocessing, deployment infrastructure, database design, and CI/CD pipelines.
- **Ömer Faruk Bayram:** POWSM-CTC model evaluation and experiments, admin analytics dashboard development, and signal processing research.

Both team members contributed to documentation, testing, and project planning.

Chapter 3

RELATED WORK

3.1 Speech Foundation Models and Phonetic Transcription

Several open-source deep learning models are available for phonetic transcription. Wav2Vec 2.0 [1] and WavLM [2] provide self-supervised speech representations that can be fine-tuned for phonetic recognition, though they require significant adaptation. For this application, POWSM (Phonetic Open Whisper-Style Speech Model) [12] is best suited, as it is specifically engineered for phonetic transcription with textual transcription context. The model is open to fine-tuning, which is valuable for adapting to Turkish-native English speakers. Connectionist Temporal Classification (CTC) [10] is commonly used for sequence alignment in phonetic transcription tasks.

3.2 CTC-Based Speech Models

Connectionist Temporal Classification (CTC) [10] is widely used for sequence alignment in speech processing. OWSM-CTC [15] is an encoder-only variant of the Open Whisper-Style Speech Model that uses CTC loss instead of an encoder-decoder architecture. With 1 billion parameters trained on 180,000 hours of multilingual speech data, OWSM-CTC achieves 3–4× faster inference than its encoder-decoder counterpart while supporting multilingual ASR, speech translation, and language identification. The non-autoregressive nature of CTC models makes them attractive for real-time applications; however, the lack of a decoder limits their ability to perform text-conditioned tasks such as Grapheme-to-Phoneme (G2P) conversion, which is essential for pronunciation assessment pipelines.

3.3 Forced Alignment and Assessment

The Montreal Forced Aligner (MFA) [14] is an industry-standard tool for phoneme-level time alignment, supporting English out-of-the-box. Pronunciation assessment employs metrics such as Phoneme Error Rate (PER) and goodness of pronunciation (GOP) scores [17] to evaluate speech quality.

3.4 L2 Speech Corpora and the Annotation Gap

A model that *detects* mispronunciations must be trained on what the learner actually *produced*, not on the dictionary form of the target word—otherwise it learns to normalize deviations away rather than surface them. This bias originates upstream, in how phonetic foundation models are built: they are trained on *canonical* phonetic transcriptions, and POWSM [12] in particular is trained on the IPAPack++ corpus, whose labels are canonical. As is typical of phonetic models, POWSM therefore emits dictionary-form phones by default—precisely the normalizing behaviour a deviation detector must overcome. This requirement exposes a real gap in publicly available L2-English data. Most resources provide either canonical phoneme transcriptions or holistic proficiency/accuracy *scores*; neither supplies the produced (perceived) phone sequence a deviation detector needs as a supervision target. L2-ARCTIC [19] is the rare corpus that annotates both canonical (CPL) and perceived (PPL) phones on identical audio—the property our ablation in Section 4.2.2 exploits—but it covers Arabic, Hindi, Korean, Mandarin, Spanish, and Vietnamese first languages, with *no Turkish-native speakers*. speechocean762 [18] contributes expert phoneme-accuracy scores at scale, but again provides scores rather than a produced transcription and contains no Turkish data.

To our knowledge, no public corpus offers perceived phonetic annotation of Turkish-native English speech. Our project's Turkish data originates from a rich and carefully constructed in-house corpus of Turkish-native English speech, kindly contributed by a collaborating linguist.[1] Our *first* fine-tuning adapter was trained directly on this corpus. It did not improve mispronunciation detection—not for any want of quality in the data, which is rich and linguistically valuable, but because the issue was one of *fit*: its phonetic annotation follows a canonical, dictionary-oriented convention with a diphthong/glide notation that differs from POWSM's own perceived output convention. As our ablation later confirms (Section 4.2.2), training a deviation detector on canonical-style labels reinforces normalization

[1] Turkish-native English speech corpus contributed by Öğr. Gör. Dr. Kardelen Kılınç, Eskişehir Technical University, School of Foreign Languages. We gratefully acknowledge this contribution.

rather than teaching it to surface what the learner produced. This experience is precisely what motivated the annotation-convention focus of the present work.

For the present version we therefore prepared a small set re-annotated with the phones actually produced, in the model's own convention: four Turkish-native speakers reading thirteen sentences (approximately 52 clips). The set is deliberately small—too small to train a model from scratch—so it is used in two complementary roles: mixed with L2-ARCTIC as in-domain fine-tuning data (the leave-one-speaker-out folds of Section 4.2.2), and as the held-out validation set against which both the baseline and fine-tuned models are measured (Section 8.2). This dual use lets a small, carefully annotated set do real work despite the absence of a large public Turkish-L2 resource.

3.5 Commercial Applications

Commercial pronunciation assessment platforms demonstrate practical applications of these technologies. ELSA Speak [3] provides real-time pronunciation feedback for English language learners with personalized AI coaching. Pronounce AI [16] offers instant speech feedback, pronunciation practice, and AI speaking partners for professionals and language learners, supporting both American and British English accents.

Chapter 4

METHODOLOGY

4.1 Application Architecture

Nounce is built as an industry-standard web-based application with a focus on user experience. The full-stack architecture is dockerized for consistent deployment and development environments. The frontend utilizes React with TanStack Start for server-side rendering and routing, providing a modern, responsive user interface built with Shadcn UI components for a sleek, accessible design. The backend leverages TanStack Start's server functions for API endpoints and business logic.

Authentication is handled through Clerk (free tier), supporting email/password and social sign-in options (Google, GitHub). Authorization is implemented using user metadata stored in Clerk, enabling role-based access control for administrative features.

Data persistence is handled through Neon, a serverless PostgreSQL database, providing scalable and reliable storage for user recordings and metadata with automatic scaling and backups. Audio file storage is implemented using Cloudflare R2 object storage solution. The entire stack is containerized using Docker, enabling reproducible deployments across development, staging, and production environments.

Modern server state management and caching are implemented using TanStack Query, which provides efficient data fetching, automatic background refetching, and intelligent caching mechanisms. This ensures optimal performance and user experience by minimizing unnecessary network requests and maintaining responsive UI updates.

4.1.1 Production Deployment Architecture

The production deployment follows a serverless, cloud-native architecture:

Domain and DNS: The application is served at `https://nounce.pro`, with the domain purchased and configured for production use. DNS management is handled through Cloudflare, which provides reliable DNS resolution and domain management capabilities.

Application Hosting: The TanStack Start application is deployed on Fly.io, providing global edge deployment with automatic scaling based on traffic. The application is containerized using Docker and deployed through automated CI/CD pipelines. The application is accessible via the `nounce.pro` domain, with Cloudflare's global network providing DDoS protection and enhanced security.

Security and Protection: Cloudflare's global network provides comprehensive DDoS protection, protecting the application from distributed denial-of-service attacks. The Cloudflare infrastructure also offers additional security features including SSL/TLS termination, rate limiting, and web application firewall capabilities.

Database: Neon serverless PostgreSQL provides the database layer with automatic scaling, point-in-time recovery, and branching capabilities for development and testing.

ML Services: Machine learning inference is handled by a single RunPod serverless GPU endpoint:

- **Assessment Endpoint:** Handles pronunciation assessment end to end with POWSM [12]—CTC phone recognition and forced alignment, audio-quality abstention, feature-vector difference, and GOP scoring. (The prototype's separate G2P/IPA-generation endpoint and Montreal Forced Aligner were removed; see Section 2.1.1.)

Model Caching: RunPod network volumes cache the POWSM [12] model (and the fine-tuned adapter weights) to reduce cold-start time; subsequent requests reuse the cached model.

Reference Speech: Reference pronunciations are human native recordings in General American and Received Pronunciation, precomputed into phone sequences offline; the prototype's ElevenLabs text-to-speech generation was retired.

Deployment Automation: GitHub Actions CI/CD pipelines handle building Docker images, running tests, linting, and automated deployment to Fly.io. The build process includes diagram generation from PlantUML source files, ensuring documentation stays synchronized with code changes.

4.2 Machine Learning Approach

The machine learning pipeline follows an experimental and iterative approach. After evaluating multiple open-source phonetic transcription models, including

Wav2Vec 2.0, WavLM, and POWSM [12], POWSM was selected as the best-performing solution for the specific use case. The selection was based on accuracy, inference speed, and compatibility with the target user population (Turkish-native English speakers).

POWSM [12] is integrated for both Phone Recognition (PR) and Grapheme-to-Phoneme (G2P) tasks, providing accurate phonetic transcriptions. The implementation required extensive experimentation and custom implementation due to the lack of comprehensive documentation and standardized inference providers for many open-source phonetic transcription models. Beyond the baseline model, we fine-tuned POWSM on L2 speech to improve mispronunciation detection; the design of that experiment, which centers on the *annotation convention* used as the training target, is described in Section 4.2.2 and evaluated in Section 8.2.

4.2.1 POWSM-CTC Variant Evaluation

As part of the model selection process, the CTC variant of the OWSM architecture (`espnet/owsm_ctc_v3.1_1B`) [15] was evaluated as an alternative to the encoder-decoder POWSM model. The CTC variant offers several theoretical advantages: 3–4× faster inference due to its non-autoregressive nature, native support for batch decoding, inherent CTC-based forced alignment capability, and a larger training set of 180,000 hours compared to POWSM's 17,000 hours.

However, the evaluation revealed critical limitations for the pronunciation assessment use case. First, an architectural incompatibility was discovered between the ESPnet library version installed via PyPI and the upstream checkpoint format, causing the convolutional subsampling layer (`Conv2dSubsampling`) to load with misaligned weight keys. A compatibility patch was developed to restructure the layer before model loading, but this introduced additional complexity and potential for subtle errors. Second, the CTC model uses a different phone inventory that strips diacritics, making cross-model comparison with the encoder-decoder POWSM unreliable. Most critically, the CTC architecture lacks a decoder, which is essential for text-conditioned G2P conversion—the G2P task returned empty transcriptions for test samples, causing all Phone Recognition outputs to be classified as insertion errors (averaging 86.7 errors across 3 test recordings).

Based on these findings, the encoder-decoder POWSM was retained as the production model for both PR and G2P tasks, providing a consistent phone inventory and reliable text-conditioned phonetic transcription.

4.2.2 Fine-Tuning: Perceived vs. Canonical Annotation

A pronunciation-feedback system must transcribe *what the learner actually produced*, not normalize it back to the dictionary form—otherwise it cannot detect

a mispronunciation at all. Our central hypothesis is therefore an *annotation-convention* claim: valid mispronunciation-detection fine-tuning requires **perceived** (produced) phone labels in the model's own output convention, and training on **canonical** (dictionary) labels actively harms the detector by reinforcing normalization.

We test this with a controlled, same-data, opposite-supervision ablation on the L2-ARCTIC corpus [19], which ships, for each utterance, both a canonical phoneme transcription (CPL) and a perceived transcription (PPL) annotated on identical audio. We fine-tune parameter-efficient PEFT LoRA adapters [11] on the frozen POWSM encoder (rank 32, α=64, attention projections only), holding everything fixed except the label target:

- `l2a_cpl` — canonical (CPL) targets (the control: should reproduce the normalization failure on clean data).
- `l2a_ppl` — perceived (PPL) targets (the corrected condition).
- `l2a_ppl_dora` — a DoRA [13] variant of the perceived condition (a LoRA-vs-DoRA secondary ablation).
- `l2a_ppl_tr_fold1--4` — L2-ARCTIC PPL plus three of the four annotated Turkish speakers, in a Leave-One-Speaker-Out (LOSO) design, to measure speaker-independently whether a small amount of in-domain Turkish data helps.

Because dev loss is not comparable across different label distributions, models are compared only on held-out behaviour: phone error rate (PER) and substitution recall on a Turkish-native held-out set, deviation recall on L2-ARCTIC, and a native false-positive (drift) rate that a deployable adapter must not inflate. Each adapter was trained for 30 epochs and, after observing that dev loss had not plateaued, re-trained for 60 epochs (`l2a_*_long`); the longer schedule is what sharpens the result (Section 8.2). All training and evaluation code, and the adapter artifacts, are available in the project repository [8].

[TODO: Add a paragraph describing the collaborator's independent L2-ARCTIC fine-tune (recipe, data split, target convention) for a head-to-head comparison once their results are available.]

4.3 Development Methodology

The project employed parallel development tracks, leveraging prior professional software development expertise while building new knowledge in signal processing

and deep learning domains. Signal processing components, including audio preprocessing, quality assessment, and feature extraction, were developed alongside deep learning model integration and assessment pipelines.

Experimentation and prototyping were conducted using Python notebooks (Jupyter/IPython), enabling rapid iteration on digital signal processing (DSP) techniques, visualization of audio features, and deep learning model evaluation. These notebooks served as both development tools and documentation of the experimentation process, capturing insights and learnings that informed the final implementation.

The development process emphasizes modularity and separation of concerns, with clear boundaries between frontend, backend, database, and machine learning components. This architecture facilitates independent development and testing of each component while maintaining system integration.

4.3.1 Local Development and RunPod Simulation

For local development, a custom RunPod serverless simulator was implemented to replicate the production RunPod API behavior without requiring cloud GPU resources. The simulator architecture consists of:

Proxy Server: A FastAPI-based proxy server (`mod/dev/runpod_proxy.py`) that mimics the RunPod Cloud API, providing endpoints for job submission (`POST /v2/{endpoint_id}/run`) and status polling (`GET /v2/{endpoint_id}/status/{job_id}`). The proxy maintains in-memory job state and manages worker queues per endpoint.

Worker Containers: Docker containers for assessment and IPA generation workers, running the same code as production endpoints but accessible locally via Docker Compose networking.

Job Processing: Background async worker loops pull jobs from per-endpoint queues, ensuring sequential processing that mimics single GPU pod behavior. The simulator handles retries, timeouts, and webhook delivery, providing a faithful representation of RunPod's serverless architecture.

Configuration: Worker endpoints are mapped via `WORKER_MAP` environment variable, allowing flexible configuration of endpoint-to-worker mappings. The simulator is orchestrated using Docker Compose (`docker-compose.dev.yml`) and can be started using the `scripts/runpod.py` utility script.

This local simulation environment enables rapid development and testing of ML pipeline integration without incurring cloud GPU costs or dealing with cold-start delays during development iterations.

Chapter 5

REQUIREMENTS SPECIFICATION

5.1 Functional Requirements

5.1.1 User Authentication and Authorization

- **FR-1.1: User Registration** The system shall allow new users to register accounts with email and password authentication or social sign-in (Google, GitHub, etc.) through Clerk authentication service.
- **FR-1.2: User Login** The system shall provide secure login functionality for registered users through Clerk authentication service.
- **FR-1.3: Session Management** The system shall maintain user sessions and support automatic session expiration after periods of inactivity.
- **FR-1.4: User Profile** The system shall associate all recordings and progress data with authenticated user accounts.
- **FR-1.5: Admin Access** The system shall provide restricted administrative access for content management functions such as creating, editing, and deleting target texts and practice materials.

5.1.2 Speech Recording and Audio Input

- **FR-2.1: Browser-Based Recording** The system shall enable users to record speech directly through the web browser using the device microphone.
- **FR-2.2: Real-Time Audio Visualization** The system shall display real-time waveform visualization during recording to provide visual feedback.

- **FR-2.3: Recording Controls** The system shall provide start, stop, and pause controls for audio recording.
- **FR-2.4: Audio Format Support** The system shall accept audio recordings in WebM/Opus format from browsers and convert them to WAV format (16-bit, 16kHz, mono) for processing.
- **FR-2.5: Recording Duration Limits** The system shall enforce reasonable duration limits to prevent excessive resource consumption.

5.1.3 Audio Quality Assessment

- **FR-3.1: Basic Quality Check** The system shall perform basic audio quality assessment to ensure recordings meet minimum standards for analysis.
- **FR-3.2: Quality Feedback** The system shall provide feedback to users when recordings fail quality checks, suggesting improvements when possible.

5.1.4 Phonetic Analysis and Transcription

- **FR-4.1: Phonetic Transcription** The system shall generate International Phonetic Alphabet (IPA) transcriptions of user speech using acoustic ML models.
- **FR-4.2: Prosody Independence** The system shall produce phonetic transcriptions that are independent of prosodic variations (stress, intonation, rhythm).
- **FR-4.3: Phone-Level Alignment** The system shall align phonetic transcriptions with audio timestamps at the phone level.
- **FR-4.4: Target Comparison** The system shall compare user phonetic transcriptions against canonical IPA transcriptions of target pronunciations.
- **FR-4.5: Error Classification** The system shall classify pronunciation errors into categories: substitution, deletion, and insertion.

5.1.5 Pronunciation Feedback and Assessment

- **FR-5.1: Error Identification** The system shall identify and highlight specific phoneme-level pronunciation errors.
- **FR-5.2: Error Visualization** The system shall present pronunciation errors in a clear format showing the target vs. actual pronunciation.

- **FR-5.3: Overall Score** The system shall calculate and display an overall pronunciation accuracy score based on error rate.

5.1.6 Educational Content and Phonology Guidance

- **FR-6.1: Basic Guidance** The system shall provide basic guidance on English phonology concepts relevant to identified errors.
- **FR-6.2: Target Text Management** The system shall allow administrators to create, edit, and manage target texts for pronunciation practice.

5.1.7 Progress Tracking and History

- **FR-7.1: Recording History** The system shall maintain a history of user recordings with timestamps and metadata.
- **FR-7.2: Basic Progress Tracking** The system shall allow users to view their recording history and basic progress information.

5.1.8 Data Storage and Management

- **FR-8.1: Audio Storage** The system shall store processed audio files (16-bit, 16kHz, mono WAV) in persistent storage for long-term access.
- **FR-8.2: Metadata Storage** The system shall store recording metadata (user ID, file path, timestamps, analysis results) in a PostgreSQL database.

5.1.9 Asynchronous Processing

- **FR-9.1: Job Queue** The system shall process audio analysis tasks asynchronously using a job queue system to handle concurrent requests.
- **FR-9.2: Processing Status** The system shall provide status updates to users during audio processing.
- **FR-9.3: Error Handling** The system shall handle processing failures gracefully, providing clear error messages.

5.1.10 User Interface and Experience

- **FR-10.1: Responsive Design** The system shall provide a responsive user interface that works effectively on desktop and mobile devices.

- **FR-10.2: Intuitive Navigation** The system shall provide clear navigation and workflow for recording and viewing results.
- **FR-10.3: Visual Feedback** The system shall provide immediate visual feedback for user actions, including loading states and error notifications.
- **FR-10.4: Language Support** The system interface shall clearly communicate that it supports English with US accent (potentially UK), targeting Turkish-native speakers, and that other languages are not supported.

Chapter 6

DESIGN

6.1 Information Structure

6.1.1 Entity-Relationship Diagram

The entity-relationship diagram represents the complete database schema for Nounce. All tables and relationships shown in the diagram are fully implemented and operational, including:

- `practice_texts` - Practice materials with difficulty and category classification
- `authors` - Reference voice/author management
- `reference_speeches` - Target pronunciation audio files
- `user_recordings` - User-submitted audio recordings
- `audio_quality_metrics` - Quality assessment results
- `analyses` - Pronunciation assessment results
- `alignments` - Phone-level time alignment data
- `phoneme_errors` and `word_errors` - Error tracking
- `jobs` and `ipa_generation_jobs` - Asynchronous job tracking
- `user_preferences` - User settings

The schema supports the complete ML pipeline and assessment workflow, with proper indexing for efficient queries and relationships ensuring data integrity.

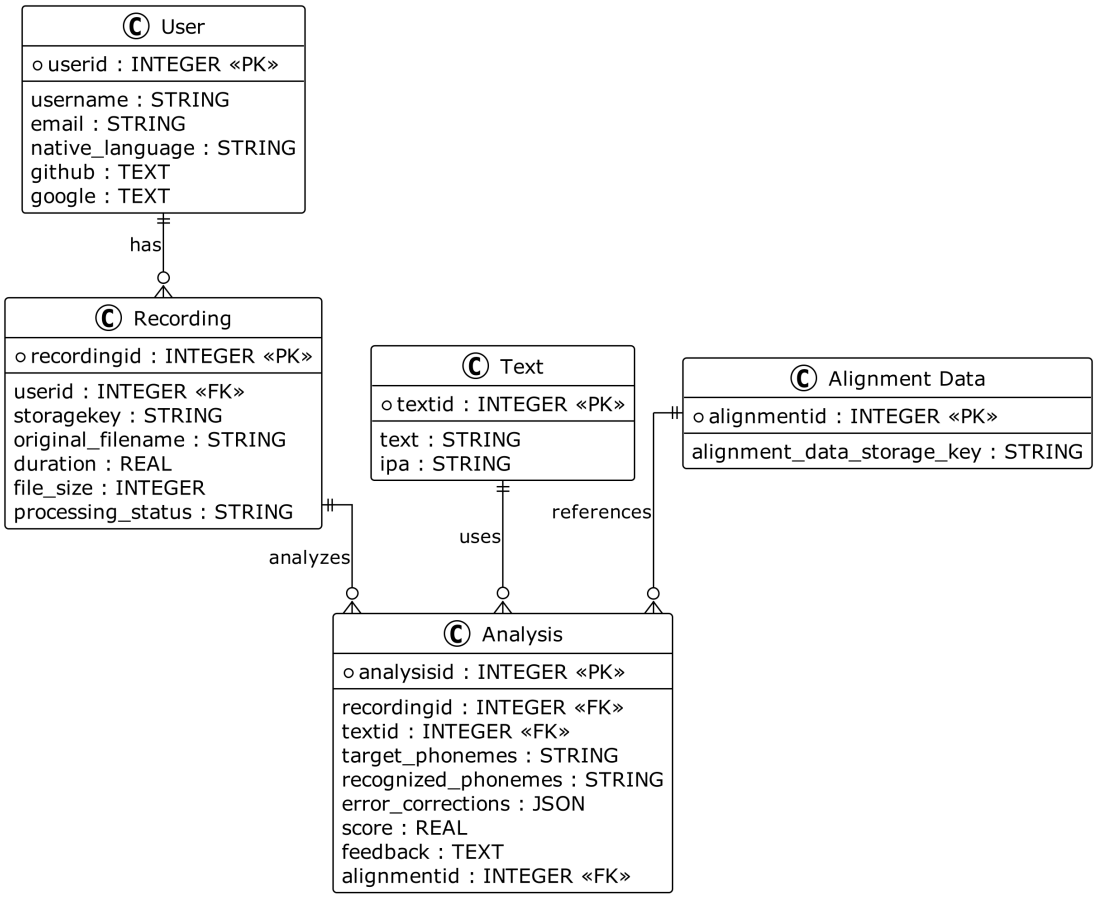

Figure 6.1: Entity-Relationship Diagram for Nounce (Planned Schema)

6.2 Information Flow

6.2.1 Pronunciation Assessment Workflow

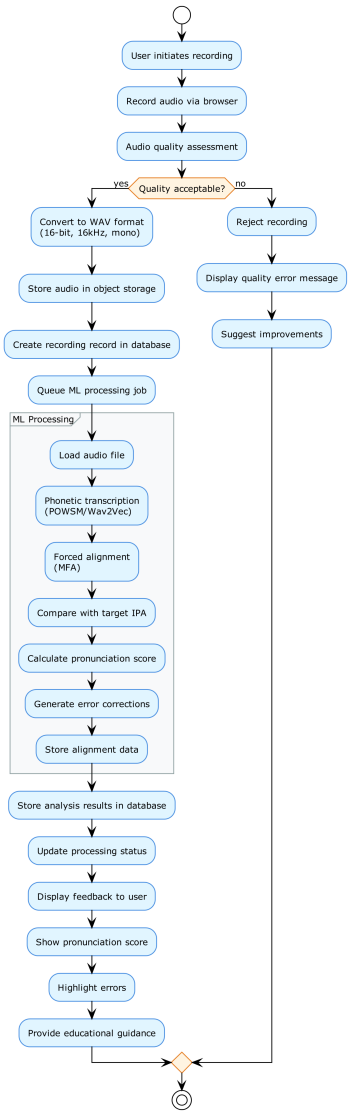

Figure 6.2: Activity Diagram: Pronunciation Assessment Workflow

6.3 System Design

6.3.1 System Architecture Diagram

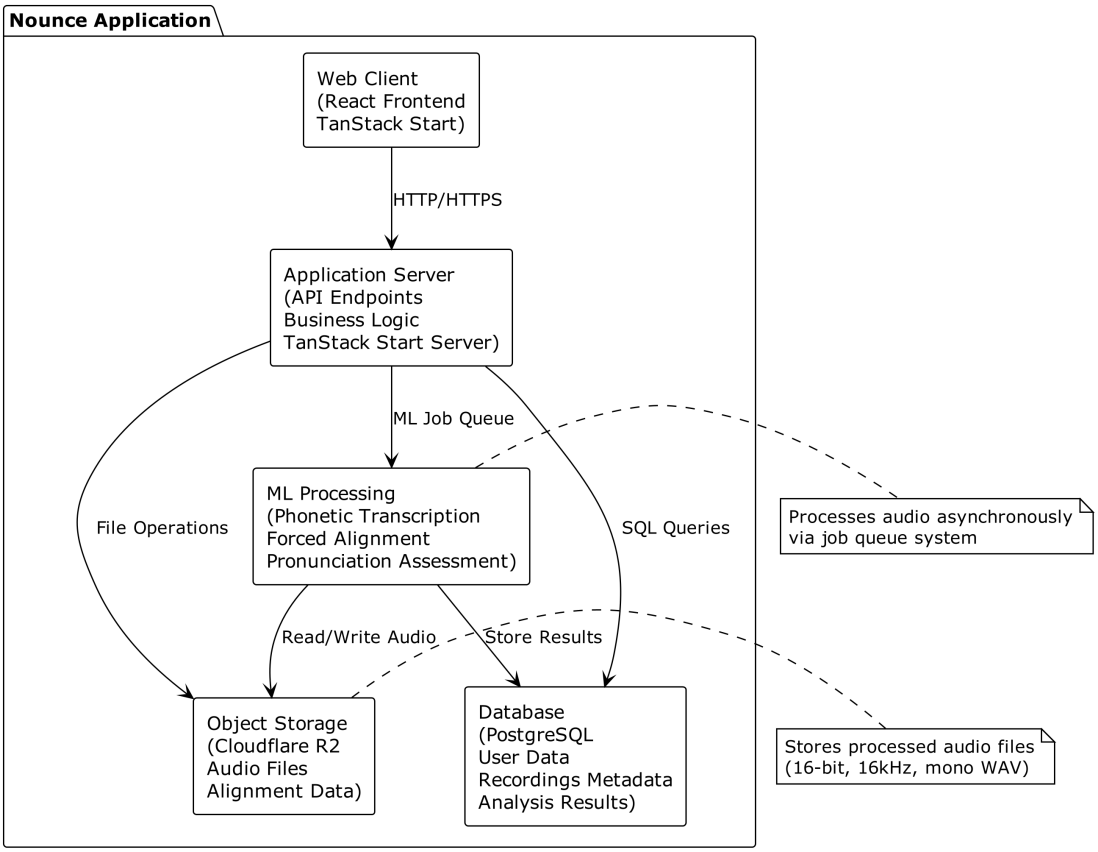

Figure 6.3: System Architecture Diagram for Nounce

6.3.2 Deployment Architecture Diagram

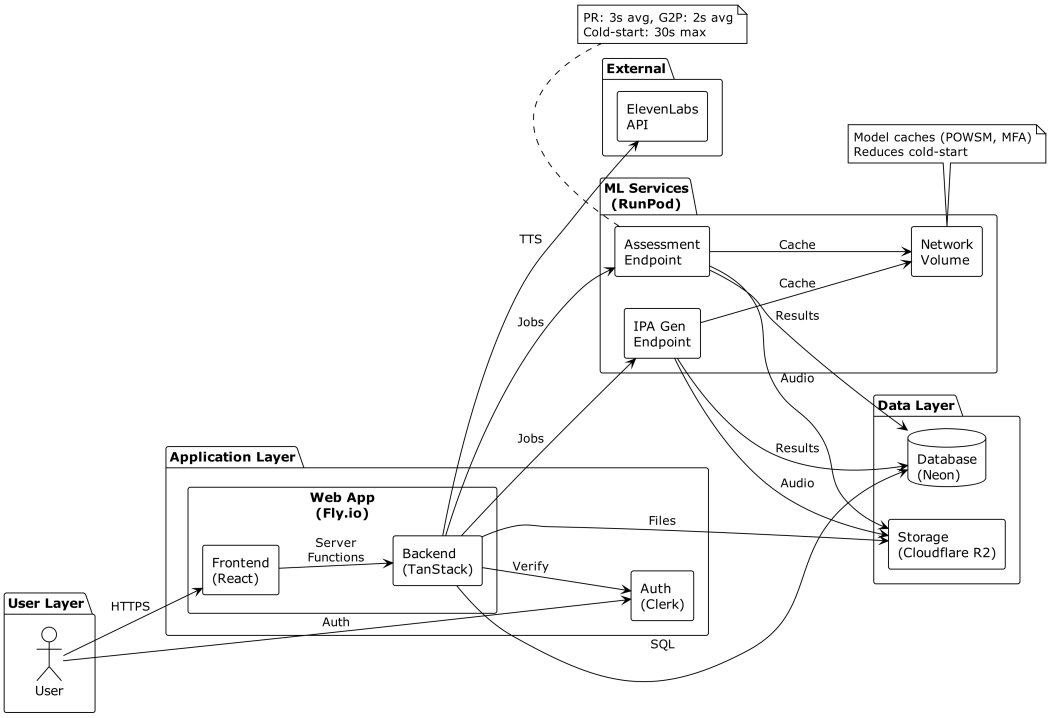

Figure 6.4: Production Deployment Architecture

6.3.3 RunPod Simulator Architecture Diagram

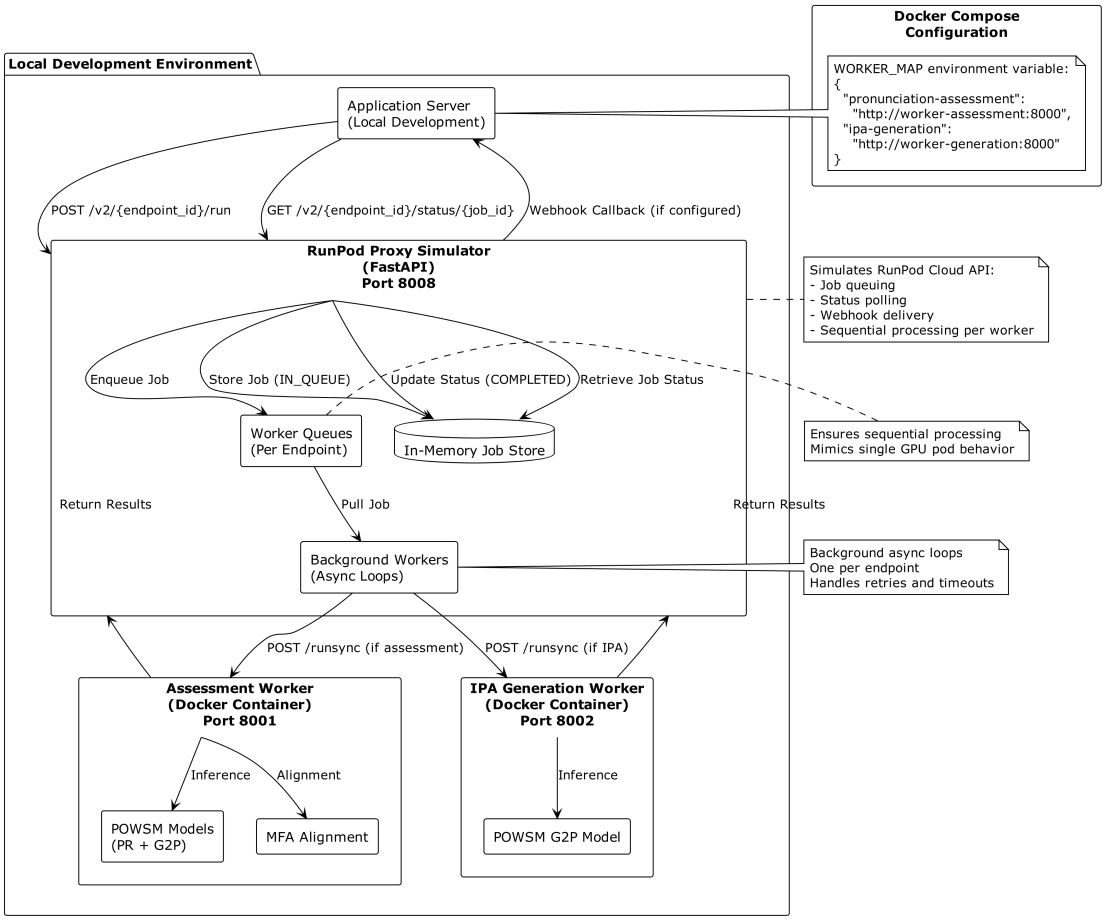

Figure 6.5: Local RunPod Serverless Simulator Architecture

6.4 User Interface Design

6.4.1 Application Logo

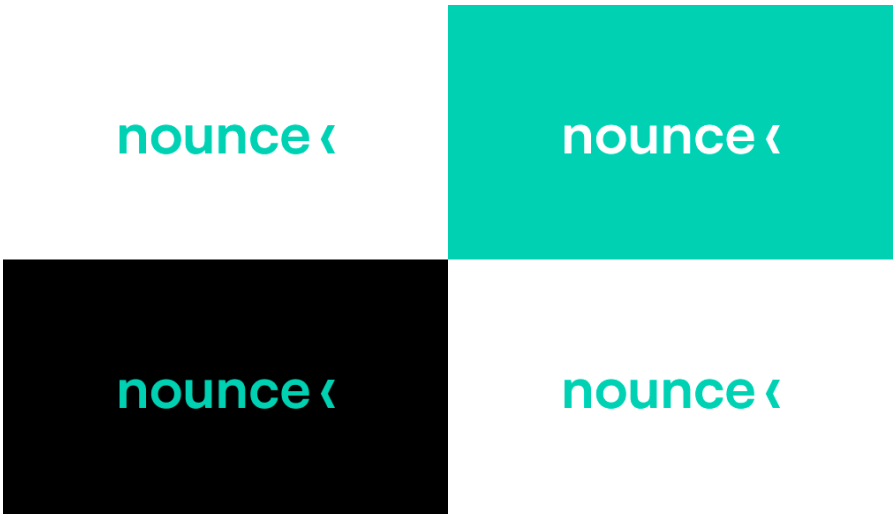

Figure 6.6: Nounce Application Logo

6.4.2 User Interface Screenshots

The Nounce application provides a modern, intuitive user interface built with Shadcn UI and ElevenLabs UI components. The following screenshots demonstrate key features and workflows of the application. All screenshots are captured in dark mode.

Home Page

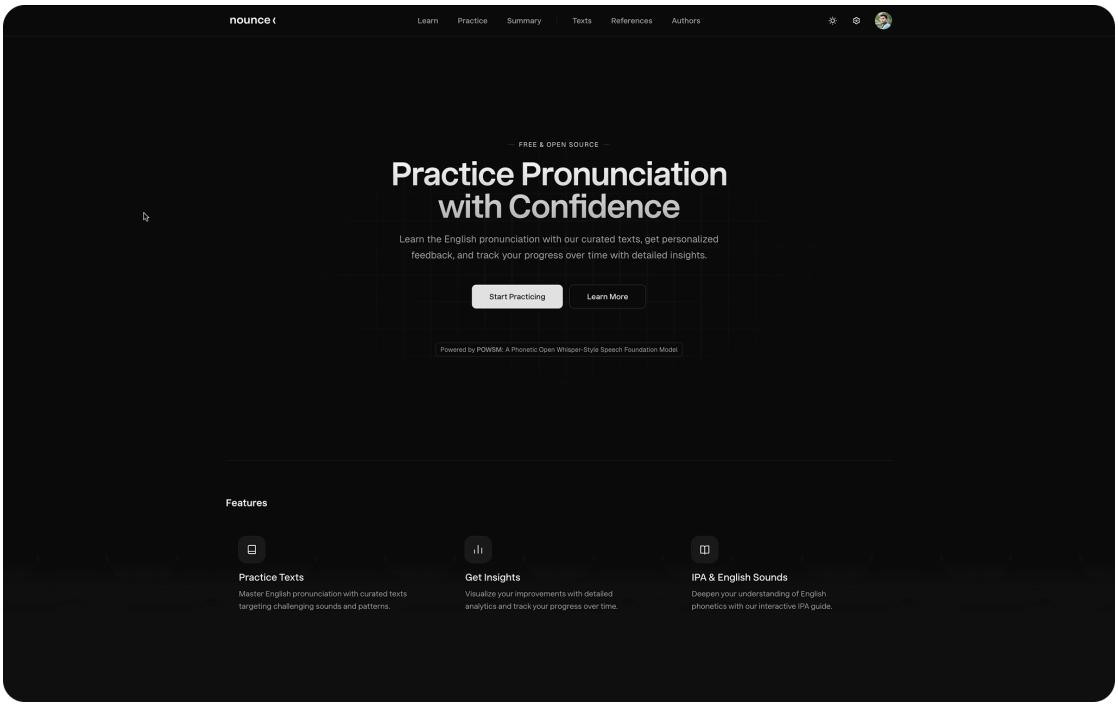

Figure 6.7: Home Page - Main landing page with feature overview and navigation

Practice Text Selection

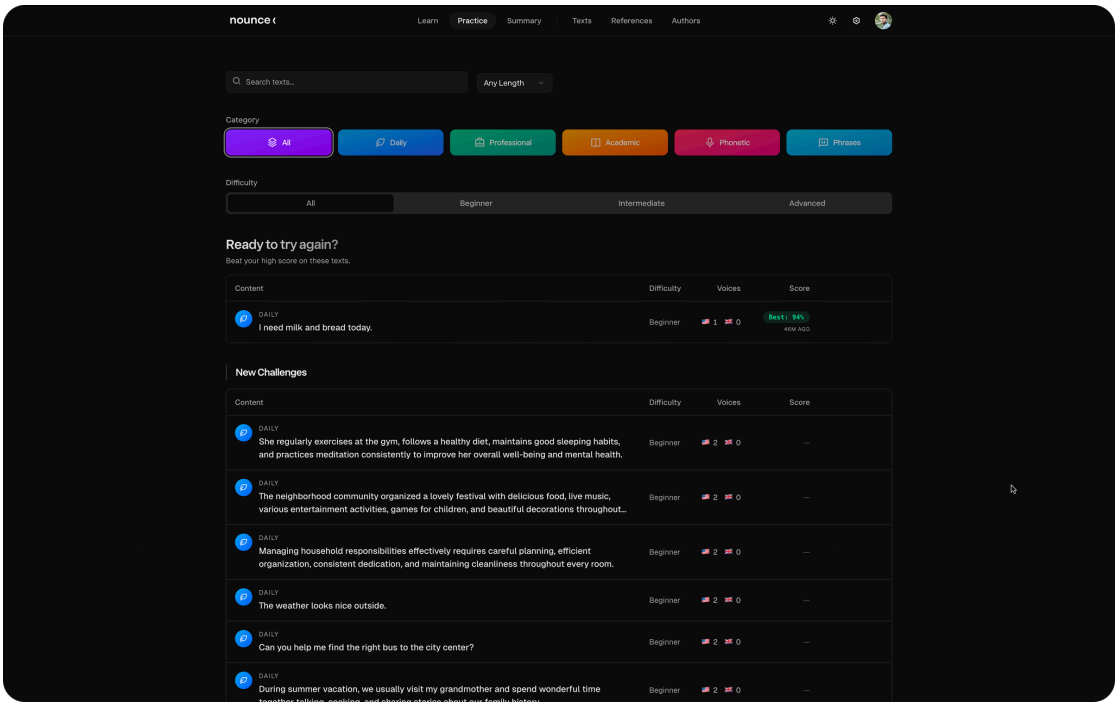

Figure 6.8: Practice Text Selection - Browse and filter practice texts by difficulty and category

Recording Interface

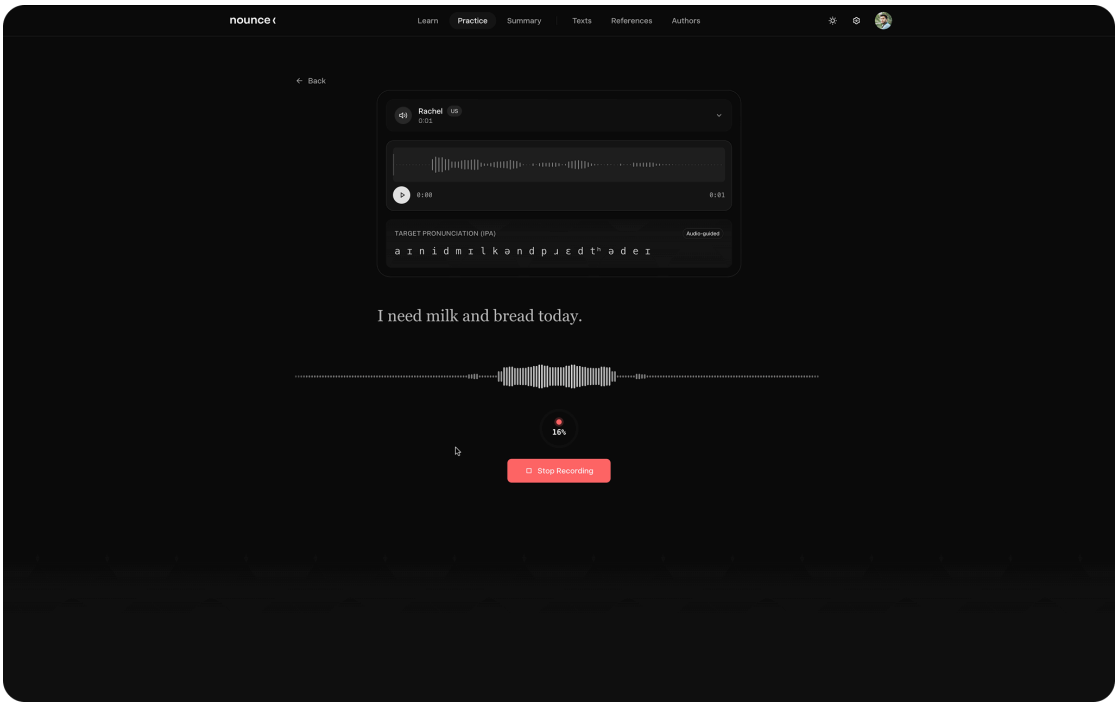

Figure 6.9: Recording Interface - Browser-based audio recording with real-time waveform visualization (dark mode)

Analysis Results

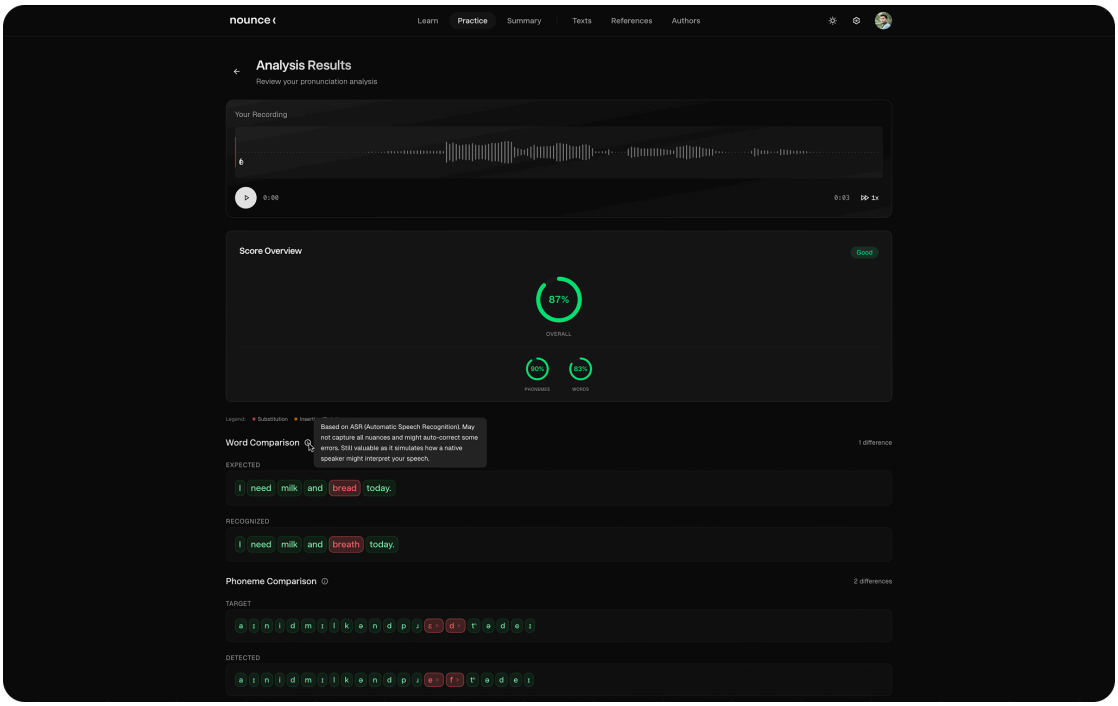

Figure 6.10: Analysis Results - Detailed pronunciation feedback with phoneme and word-level error visualization

Progress Dashboard

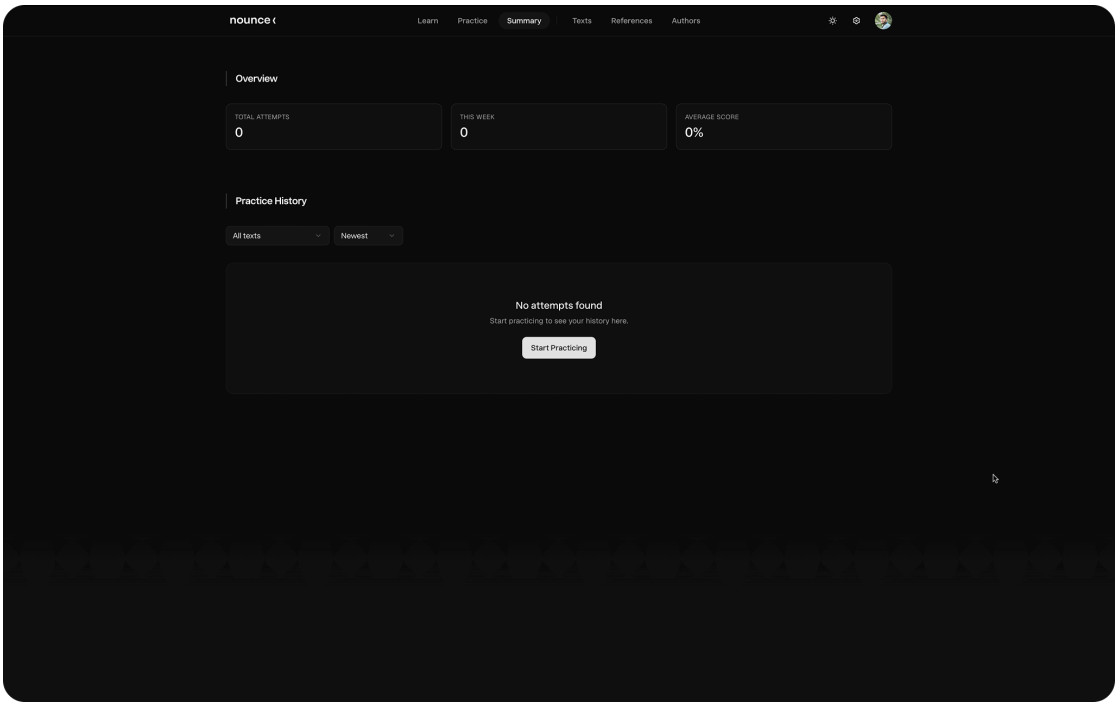

Figure 6.11: Progress Dashboard - User statistics and error analysis over time

Educational Content

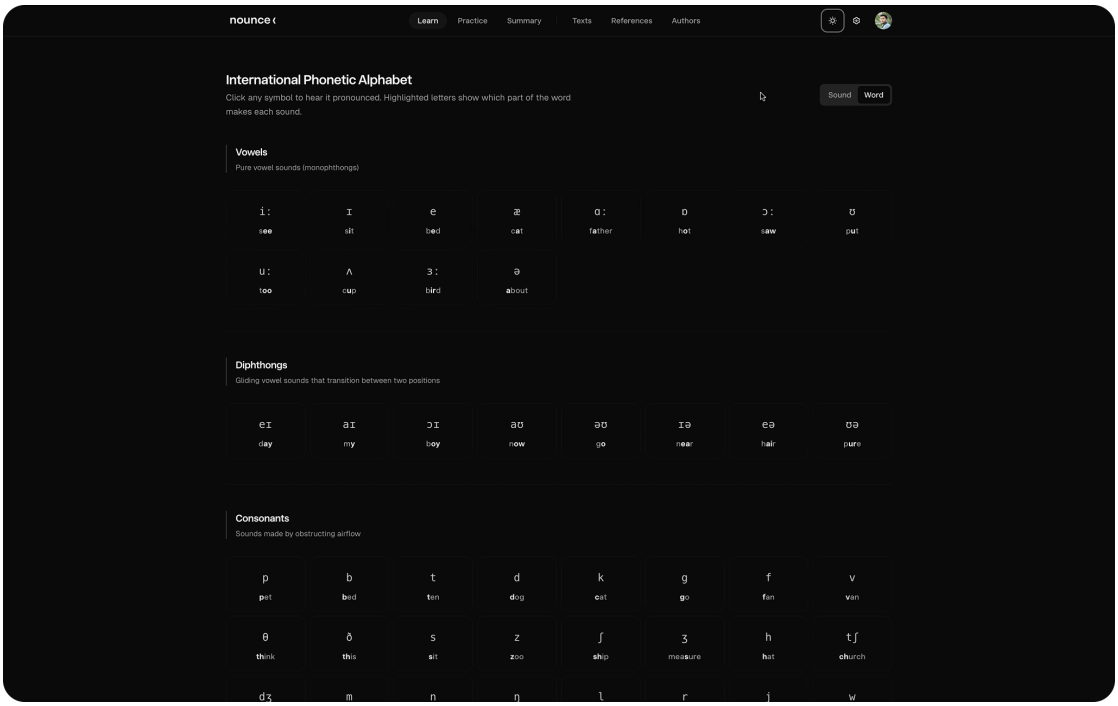

Figure 6.12: Educational Content - IPA and English phonology learning materials

Admin Interface

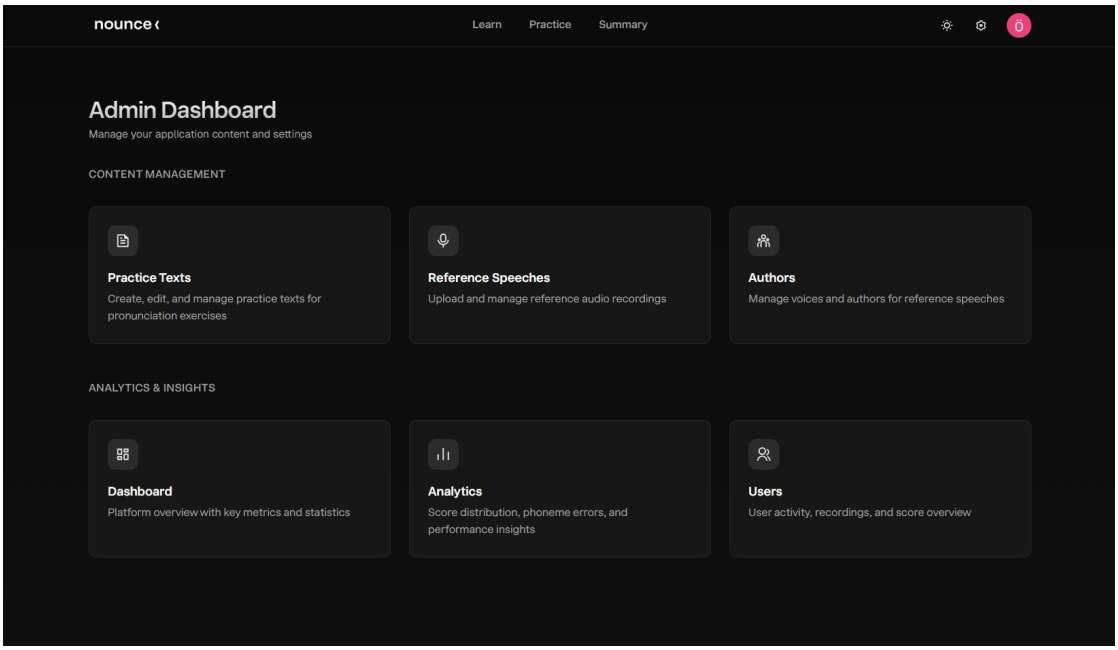

Figure 6.13: Admin Interface - Content management for practice texts and reference speeches

Mobile View

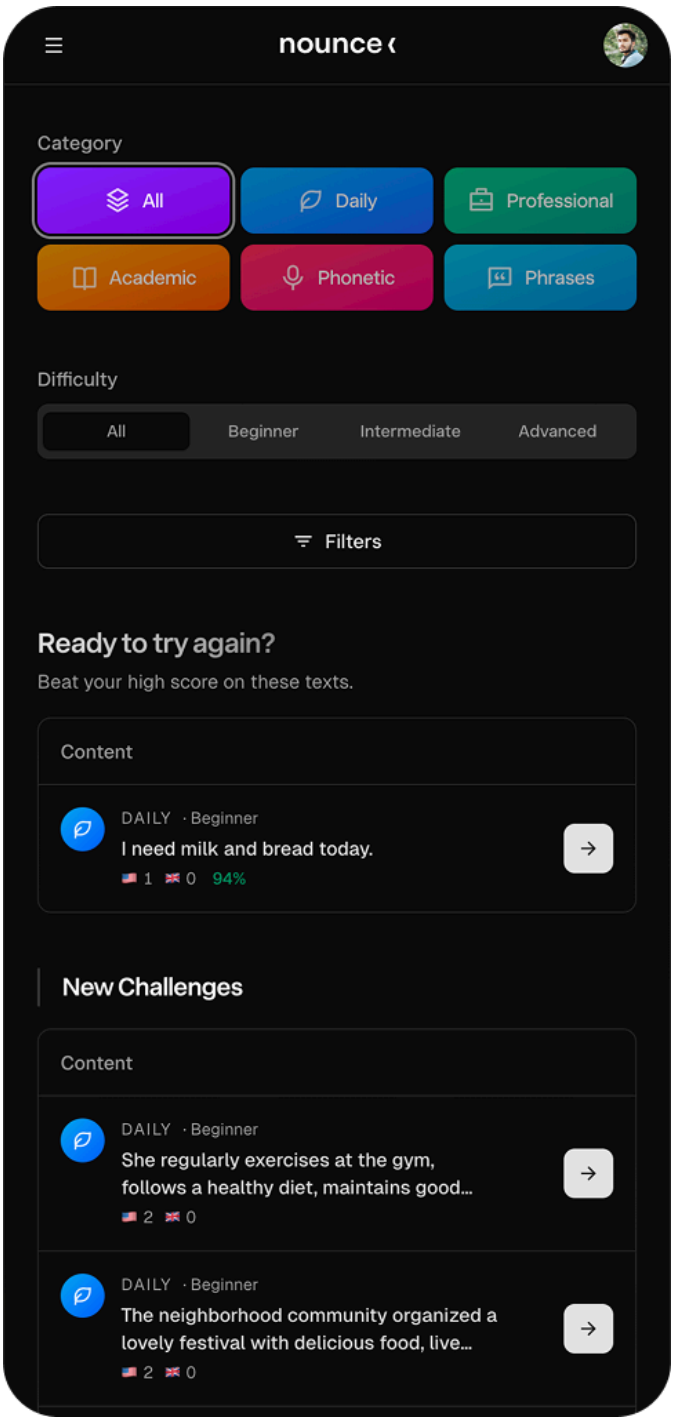

Figure 6.14: Mobile View - Practice text selection on mobile devices

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation

The implementation of Nounce follows the architecture and methodology outlined in previous chapters. The source code is organized in a structured repository [8] with clear separation between the full-stack application, machine learning services, documentation, and experimental work.

7.1.1 Full-Stack Application

The main application codebase is located in the `app/` directory [4], built with modern web technologies:

Frontend: The React-based user interface is implemented using TanStack Start for server-side rendering and routing. The UI follows modern design principles with Tailwind CSS, Shadcn UI, and ElevenLabs UI components, providing a sleek, responsive, and accessible experience across desktop and mobile devices. Key user-facing features include:

- Practice text selection with filtering by difficulty (beginner, intermediate, advanced) and category (daily, professional, academic, phonetic challenge, common phrase)
- Browser-based audio recording with real-time waveform visualization
- Audio upload functionality with format conversion
- Pronunciation analysis results in a git-style diff viewer with goodness-of-pronunciation colour-coding, plus per-phone audio playback that highlights each phone as it is spoken

- Progress tracking and summary dashboard with statistics and error analysis
- Educational content pages for IPA and English phonology

Backend: The TanStack Start backend provides server functions for API endpoints and business logic. Authentication is handled through Clerk (free tier), supporting email/password and social sign-in options (Google, GitHub). Authorization is implemented using user metadata stored in Clerk, enabling role-based access control. The application uses TanStack Query for efficient data fetching, caching, and state management.

Database: The complete database schema is implemented using Drizzle ORM with Neon serverless PostgreSQL. All planned tables are operational, including:

- `practice_texts` - Practice materials with difficulty and category classification
- `authors` - Reference voice/author management (native General American and Received Pronunciation speakers)
- `reference_speeches` - Target pronunciation audio files with IPA transcriptions
- `user_recordings` - User-submitted audio recordings with metadata
- `audio_quality_metrics` - Quality assessment results (SNR, silence ratio, clipping ratio)
- `analyses` - Pronunciation assessment results with phoneme and word-level scores
- `alignments` - Phone-level time alignment data (TextGrid files)
- `phoneme_errors` and `word_errors` - Normalized error data for aggregation
- `jobs` and `ipa_generation_jobs` - Asynchronous job tracking for RunPod endpoints
- `user_preferences` - User settings and preferences

Storage: Audio files are stored in Cloudflare R2 object storage, with processed audio files (16-bit, 16kHz, mono WAV) maintained for long-term access. Metadata is stored in the PostgreSQL database with proper indexing for efficient queries.

Admin Features: Administrative interfaces are implemented for content management:

- Practice text CRUD operations with bulk import capabilities

- Reference speech management with audio upload and IPA generation
- Author/voice management for native-speaker reference recordings
- Reference phone precomputation monitoring and management

Reference pronunciations are human native recordings in both General American and Received Pronunciation (commissioned voice talent), replacing the prototype's text-to-speech and letting learners choose a dialect to practise against.

7.1.2 Machine Learning Services

The machine learning pipeline is implemented as separate RunPod serverless endpoints in the `mod/` directory:

Assessment Endpoint: The pronunciation assessment service (`mod/assessment/`) is built around POWSM [12] and its CTC encoder, and replaces several prototype components:

- **Phone recognition:** POWSM CTC free alignment transcribes the IPA phones the learner actually produced, with real frame-level timestamps.
- **Forced alignment:** the reference phone sequence is force-aligned to the audio, also via POWSM's CTC path. This replaces the prototype's Montreal Forced Aligner and gives a single, consistent phone inventory across recognition and alignment instead of two tools with different inventories.
- **Audio-quality abstention:** a Silero voice-activity stage plus signal-quality checks (SNR, clipping) reject no-speech, noisy, or wrong-sentence recordings, returning an explained abstention rather than a misleading score.
- **Feature-aware difference:** produced and target phones are compared with an articulatory feature-vector distance (PanPhon) rather than exact-match edit distance, so a near-miss (e.g. a voicing-only swap) is treated as closer than a gross substitution. The same feature distance drives both the difference visualization and the learner-facing feedback.
- **Goodness-of-Pronunciation (GOP):** per-phone GOP and entropy are computed directly from the CTC posteriors, supporting confidence-aware, colour-coded feedback.

Because the reference phones are precomputed and supplied with each request, the GPU worker is stateless (it never re-aligns the reference). The endpoint is deployed on RunPod serverless GPU infrastructure, utilizing network volumes for model caching to minimize cold-start delays.

Decommissioned G2P endpoint. The prototype ran a separate Grapheme-to-Phoneme endpoint (`mod/ipa_generation/`) to derive target IPA from text. This was removed in the present version: reference phones are computed once, offline, with the assessment model's own alignment, eliminating a second model load and a cross-model phone-inventory mismatch that had inflated error counts.

7.1.3 Signal Processing and Experiments

Signal processing experiments and deep learning model evaluations are conducted in the `sig/exp/` directory [9], which contains Jupyter notebooks for:

- Audio preprocessing and quality assessment (SNR, silence detection, clipping detection)
- POWSM [12] model evaluation and fine-tuning experiments
- POWSM CTC forced-alignment evaluation (replacing the prototype's MFA/TextGrid pipeline)
- Cross-dataset evaluation and comparison (L2-ARCTIC, speechocean762, and the in-house Turkish set)
- Visualization of phonetic features and alignment results

These notebooks document the experimentation process and serve as reference implementations for the production pipeline.

7.1.4 Admin Analytics Dashboard

An administrative analytics dashboard was developed to provide platform-wide insights and monitoring capabilities. The dashboard consists of three pages accessible from the admin interface:

Overview Page: Displays key platform metrics including total registered users, total analyses with status breakdown (completed, pending, processing, failed), practice text counts by difficulty level, reference speech count, platform-wide average pronunciation score, and audio quality distribution across accept, warning, and reject categories.

Analytics Page: Provides deeper analytical views using interactive charts built with the Recharts library. Features include a score distribution histogram showing the spread of pronunciation scores across five ranges, an area chart showing analysis activity over the past 30 days, ranked tables of the most common phoneme errors and phoneme confusion pairs (expected vs. actual), and processing time statistics (average, median, minimum, maximum).

User Activity Page: Presents a paginated table of all users with their recording counts, analysis counts, average scores, and last active timestamps, enabling administrators to monitor user engagement.

The dashboard is built using the same technology stack as the rest of the application: Drizzle ORM aggregation queries for data retrieval, TanStack Server Functions for the API layer, and TanStack Query for client-side data fetching and caching.

7.1.5 User Progress and Summary Page

A user-facing progress dashboard was implemented to help learners track their pronunciation improvement over time. The summary page provides:

- **Statistics Overview:** Cards displaying total practice attempts, weekly attempt count with week-over-week progress percentage, and overall average pronunciation score.
- **Most Challenging Sounds:** An aggregated view of the user's most frequently missed phonemes, helping learners focus their practice on specific problem areas.
- **Practice History:** A filterable, paginated list of all practice attempts grouped by date (Today, Yesterday, Previous), with each entry showing the practice text, score, and timestamp. Users can filter by specific texts and sort by date or score.

The page requires authentication and displays a sign-in prompt for unauthenticated visitors.

7.1.6 Documentation and Deployment

The repository includes comprehensive documentation in the `doc/` directory, with learning materials, technical notes, database schema documentation, and project reports. The main README file [7] provides an overview of the project structure, setup instructions, and development guidelines. Environment variables and configuration are documented to ensure reproducible deployments across different environments [5].

The report build process is automated through `doc/report/build.sh`, which handles LaTeX compilation, bibliography processing, and diagram generation from PlantUML source files. The script integrates with `generate-diagrams.sh` to automatically generate all architecture and design diagrams before PDF compilation, ensuring documentation stays synchronized with code changes.

7.2 Deployment

Nounce is containerized using Docker, ensuring consistency across development, staging, and production environments. The deployment process is automated through GitHub Actions CI/CD pipelines [6], which handle building Docker images, running linting, running tests, and generating PlantUML diagrams from source files.

7.2.1 Production Deployment

The production application is deployed on Fly.io and served at `https://nounce.pro`. DNS management is handled through Cloudflare, which also provides DDoS protection via its global network infrastructure. The database is hosted on Neon serverless PostgreSQL, offering automatic scaling, point-in-time recovery, and branching capabilities. Audio files are stored in Cloudflare R2 object storage.

Machine learning services are deployed as RunPod serverless GPU endpoints, utilizing network volumes for model caching to reduce cold-start times. The deployment architecture is illustrated in Figure 6.4.

7.2.2 Local Development

For local development, a custom RunPod serverless simulator replicates the production API behavior without requiring cloud GPU resources. The simulator architecture, illustrated in Figure 6.5, consists of a FastAPI proxy server that mimics RunPod's job queue API, worker containers running locally via Docker Compose, and background async loops for job processing. The simulator is orchestrated using `docker-compose.dev.yml` and can be started using the `scripts/runpod.py` utility script.

The Docker configuration includes environment variable management and health checks for container monitoring. Building instructions, environment setup, and deployment procedures are documented in the repository README [7], with the build process automated through `doc/report/build.sh` script that handles LaTeX compilation and diagram generation.

Chapter 8

RESULTS

This chapter presents the results achieved in the development of Nounce, including technical achievements, system capabilities, implementation status, and model evaluation findings.

8.1 POWSM-CTC Evaluation Results

The POWSM-CTC variant (`espnet/owsm_ctc_v3.1_1B`) was evaluated on three test recordings from two Turkish-native English speakers reading English sentences. The results are summarized in Table 8.1.

Table 8.1: POWSM-CTC evaluation results on Turkish-native English speaker recordings.

Sample	PR Phones	G2P Phones	Total Errors	Error Types
umit12	86	0	86	86 insertions
yusuf12	83	0	83	83 insertions
umit14	91	0	91	91 insertions
Average	**86.7**	**0**	**86.7**	**100% insertions**

The G2P task produced empty transcriptions for all test samples, as the CTC encoder-only architecture lacks the decoder component required for text-conditioned phoneme generation. Consequently, all phonemes detected by Phone Recognition were classified as insertion errors, rendering the assessment scores meaningless. These results confirmed that the encoder-decoder POWSM architecture is necessary for the pronunciation assessment pipeline, where both PR

and G2P tasks must operate on a consistent phone inventory with reliable text-conditioned output.

8.2 Fine-Tuning Evaluation and Validation Study

This section presents the empirical centerpiece of the project: the annotation-convention ablation (Section 4.2.2), a human-judgment validation study, and an external benchmark validation.

8.2.1 Annotation-Convention Ablation (Canonical vs. Perceived)

Table 8.2 compares the baseline POWSM against the canonical- and perceived-label adapters at both training budgets. The clinically relevant metric is *deviation recall* on L2-ARCTIC: the fraction of annotated canonical→produced substitutions the model actually reproduces (rather than normalizing away). Higher PER/recall figures use the same phone-folding convention on both sides so the comparison reflects phone identity rather than diacritic differences.

Table 8.2: Canonical vs. perceived fine-tuning, 30 vs. 60 epochs. TR = Turkish-native held-out set; L2A = L2-ARCTIC held-out dev set. Native FPR is drift on correct native speech (lower is better).

Model	**TR-PER**	**TR sub-rec.**	**Native FPR**	**L2A PER/PPL**	**L2A dev**
base POWSM	0.435	0.033	0.003	0.240	0.17
`l2a_cpl` (30ep)	0.426	0.016	0.007	0.238	0.17
`l2a_cpl_long` (60ep)	0.400	0.008	0.027	0.217	**0.16**
`l2a_ppl` (30ep)	0.420	0.025	0.008	0.227	0.18
`l2a_ppl_long` (60ep)	**0.392**	**0.057**	0.024	**0.205**	**0.21**

The result confirms the hypothesis, and sharpens with training budget. At 30 epochs the canonical adapter is a *no-op*: `l2a_cpl` matches the baseline's deviation recall exactly (0.173), because the base model already emits canonical-leaning phones and canonical supervision teaches it nothing new. At 60 epochs the canonical adapter *actively harms*: `l2a_cpl_long` deviation recall drops *below* base ($0.163 < 0.173$) as the model normalizes harder and loses produced deviations. The perceived adapter moves in the opposite direction: `l2a_ppl_long`

reaches 0.213 deviation recall and is the only model to beat base on both Turkish PER (0.392) and Turkish substitution recall (0.057 vs. 0.033). The cpl-vs-ppl gap widens roughly four-fold between the two budgets—identical audio, opposite supervision, diverging behaviour. The cost of the longer schedule is increased native drift (FPR 0.024 vs. 0.003 for base); `l2a_ppl_long` is deployed as the production model, accepting this drift for materially better learner-deviation detection. The LOSO arm (adding three Turkish speakers) gives a modest, high-variance speaker-independent gain (mean PER 0.421, range 0.402–0.455).

[**TODO: insert figure `artifacts/eval/figs/l2arctic_cpl_vs_ppl.png` (per-L1 deviation recall, cpl vs ppl) into `doc/report/images/`.**]

8.2.2 Human-Judgment Validation

To confirm that the system's pronunciation score tracks human perception, native English raters score each Turkish-learner clip for intelligibility (0–10) through a blind rating interface (Figure 8.1), and we correlate the mean human rating against the system score via Spearman's ρ. On the first rater's submission (24 scored clips), $\rho = 0.45$, a moderate-to-strong positive correlation indicating the system meaningfully tracks human judgment.

[**TODO: app screenshot**]
Blind intelligibility-rating interface (`/intelligibility-score`): per-clip 0–10 scale, progress dots, audio playback.

Figure 8.1: The blind human-rating interface used in the validation study.

[**TODO: this $\rho = 0.45$ is preliminary (1 rater, 24 clips). Re-run `sig/analysis/validation.py` after $\geq$2 raters submit; report the final ρ, Krippendorff's α inter-rater agreement, and insert the scatter plot `doc/figures/scatter_system_vs_human.png`.**]

8.2.3 External Benchmark Validation (speechocean762)

As an independent, large-scale check, we evaluate the system's Goodness-of-Pronunciation (GOP) score against expert human phoneme-accuracy labels on the standard speechocean762 benchmark [18] (2,500 held-out utterances, 47,369 phones, Mandarin-L1). Mean GOP rises monotonically with the human accuracy label (-6.80, -4.65, -1.37 for accuracy 0, 1, 2), and Spearman's ρ is 0.21 at the phone level and 0.37 at the sentence level. This confirms, on a recognized public benchmark and a different L1, that the GOP component tracks human pronunciation judgment; it complements (rather than replaces) the Turkish-specific study.

[**TODO: insert the GOP-vs-accuracy box plot `artifacts/speechocean/figs/gop_vs_ac`**

8.2.4 Comparison with an Independent L2-ARCTIC Fine-Tune

[**TODO: a collaborator independently fine-tuned POWSM on L2-ARCTIC. Once their artifacts/metrics are available, add a head-to-head comparison (same held-out sets where possible) and a row to Table 8.2, discussing where the recipes agree/differ.**]

8.3 System Implementation Status

The Nounce pronunciation assessment system has been successfully implemented with the following components operational:

8.3.1 Full-Stack Web Application

The complete web application is functional and deployed, providing users with a comprehensive pronunciation assessment platform. The system successfully implements:

- **User Authentication:** Secure authentication through Clerk with support for email/password and social sign-in (Google, GitHub)
- **Practice Text Management:** Full CRUD operations for practice texts with filtering by difficulty and category
- **Audio Recording:** Browser-based recording with real-time waveform visualization and audio upload support
- **Reference Speech System:** Management of target pronunciations with TTS integration (ElevenLabs) and native speaker uploads

- **Pronunciation Assessment:** End-to-end workflow from recording to detailed analysis results
- **Progress Tracking:** User history, statistics, and error analysis with aggregation capabilities
- **Admin Interface:** Complete administrative tools for content and system management

8.3.2 Database Schema

The complete database schema is implemented and operational, with all planned tables, relationships, and indexes in place. The schema supports:

- User recordings with metadata and quality metrics
- Pronunciation analyses with phoneme and word-level scoring
- Error tracking with normalized data for aggregation
- Asynchronous job processing with status tracking
- Reference speech management with IPA transcriptions
- User preferences and progress data

8.3.3 Machine Learning Pipeline

The machine learning assessment pipeline is fully integrated:

- **POWSM Integration:** Successfully integrated POWSM [12] models for Phone Recognition (PR) and Grapheme-to-Phoneme (G2P) tasks
- **Phonetic Transcription:** Accurate IPA transcription from audio using state-of-the-art models
- **Error Detection:** Phoneme-level error classification (substitution, insertion, deletion) with edit distance algorithms
- **Time Alignment:** MFA integration for phone-level timestamp generation
- **Score Calculation:** Pronunciation accuracy scoring based on error rates

The assessment endpoint processes audio recordings and returns detailed analysis including:

- Actual IPA transcription from user speech
- Target IPA transcription from reference text
- Overall pronunciation score (0.0-1.0)
- Phoneme-level error list with timestamps
- Word-level error analysis

8.3.4 Audio Quality Assessment

Audio quality metrics are calculated and stored for each user recording:

- Signal-to-Noise Ratio (SNR) in decibels
- Silence ratio (percentage of quiet segments)
- Clipping ratio (percentage of clipped samples)
- Noise ratio (percentage of noise segments)
- Quality status classification (accept, warning, reject)

Quality thresholds are applied at the application layer, allowing flexible adjustment without database migrations.

8.3.5 Deployment Infrastructure

The system is deployed using modern cloud infrastructure:

- **Domain:** The application is served at `https://nounce.pro`, with the domain purchased and configured for production use
- **DNS and Security:** DNS management and DDoS protection are handled through Cloudflare's global network infrastructure
- **Application Hosting:** Docker containerized application deployed on Fly.io with global edge deployment
- **Database:** Neon serverless PostgreSQL with automatic scaling and backups
- **Object Storage:** Cloudflare R2 for audio file storage
- **ML Services:** RunPod serverless endpoints for GPU-powered inference
- **CI/CD:** GitHub Actions pipelines for automated testing and deployment

8.4 Technical Achievements

8.4.1 Architecture and Design

The system demonstrates a well-architected full-stack application with:

- Clear separation of concerns between frontend, backend, database, and ML services
- Type-safe database operations using Drizzle ORM with TypeScript
- Modern React patterns with server-side rendering and efficient data fetching
- Scalable serverless architecture supporting concurrent users
- Comprehensive error handling and user feedback mechanisms

8.4.2 User Experience

The application provides an intuitive and responsive user experience:

- Modern, accessible UI following web standards and best practices
- Real-time feedback during recording and processing
- Clear visualization of pronunciation errors with IPA highlighting
- Progress tracking with historical data and statistics
- Mobile-responsive design supporting multiple devices

8.4.3 Integration Challenges and Solutions

Several technical challenges were successfully addressed during implementation:

Model Integration: Integrating POWSM [12] models required extensive experimentation due to limited documentation. The solution involved custom implementation of model loading, inference pipelines, and output parsing.

Audio Processing: Handling various audio formats and quality levels required robust preprocessing pipelines. The system implements format conversion, quality assessment, and normalization to ensure consistent input for ML models.

Asynchronous Processing: Managing long-running ML inference tasks required a job queue system. The implementation uses RunPod's serverless architecture with status tracking and webhook callbacks for result delivery.

Time Alignment: Integrating MFA for phone-level timestamps required careful handling of transcription formats and TextGrid parsing. The system supports both MFA-based alignment and fallback timestamp estimation.

8.5 Performance and Scalability

The system architecture supports scalability and demonstrates good performance characteristics:

- Serverless database (Neon) with automatic scaling
- Object storage (Cloudflare R2) for efficient audio file management
- RunPod GPU endpoints for parallel ML inference with network volume caching
- Job queue system for handling concurrent assessment requests
- Efficient database indexing for fast queries

Performance Metrics:

- Average Phone Recognition (PR) task execution time: 3 seconds
- Average Grapheme-to-Phoneme (G2P) task execution time: 2 seconds
- Maximum observed cold-start time: 30 seconds (first request after inactivity)
- Subsequent requests benefit from cached models in network volumes, eliminating cold-start delays

These performance characteristics enable responsive user experience while maintaining cost efficiency through serverless GPU infrastructure that scales to zero when not in use.

Chapter 9

CONCLUSION

This project successfully developed Nounce, a comprehensive web-based pronunciation assessment system for English language learners. The system integrates state-of-the-art machine learning models, modern web technologies, and pedagogical principles to provide detailed, actionable feedback on pronunciation accuracy.

9.1 Project Summary

Nounce addresses the critical need for accessible pronunciation learning tools by providing:

- **Phonetic-Level Analysis:** Detailed pronunciation assessment at the phoneme level using the International Phonetic Alphabet (IPA)
- **Automated Feedback:** Instant, judgment-free feedback on pronunciation errors with clear visualization
- **Educational Integration:** Guidance on English phonology concepts to help learners understand and improve
- **Progress Tracking:** Historical analysis and statistics to monitor improvement over time
- **Accessible Design:** Web-based platform accessible from any device without requiring native speaker tutors

9.2 Technical Contributions

The project demonstrates several technical achievements:

Model Integration: Successfully integrated POWSM [12] (Phonetic Open Whisper-Style Speech Model), a state-of-the-art phonetic transcription model, for accurate phone recognition and grapheme-to-phoneme conversion. The integration required custom implementation due to limited documentation, contributing to the open-source community's understanding of these models.

Full-Stack Architecture: Developed a production-ready full-stack application using modern technologies (TanStack Start, React, TypeScript, Drizzle ORM) with a focus on type safety, scalability, and maintainability. The architecture demonstrates best practices in separation of concerns, error handling, and user experience design.

ML Pipeline: Implemented a complete machine learning pipeline combining phonetic transcription, forced alignment (MFA), and error detection algorithms. The pipeline processes audio recordings asynchronously and provides detailed analysis results with phone-level timestamps.

Database Design: Designed and implemented a comprehensive database schema supporting user recordings, analyses, error tracking, and progress data. The schema enables efficient querying and aggregation for analytics and reporting.

9.3 Educational Impact

Nounce contributes to language learning by:

- Providing 24/7 access to pronunciation assessment without requiring human tutors
- Offering detailed, phonetic-level feedback that helps learners understand specific sound production issues
- Creating a safe, judgment-free environment for practice
- Supporting self-directed learning with progress tracking and historical analysis
- Introducing learners to IPA and phonetic concepts, building domain knowledge

The system acknowledges that it does not replace human-to-human interaction in language learning, but rather serves as a supplementary tool for learners who may lack access to native speakers or formal language education resources.

9.4 Conclusion and Future Work

9.4.1 State of the Platform

All core features are functional, and the application is deployed. The system provides a complete pronunciation assessment workflow from recording to detailed feedback, with administrative tools for content management. Additional content management and data curation is still needed for future expansion.

9.4.2 Limitations

All core features are functional, and the application is deployed. Additional content management and data curation is still needed for future expansion to support a broader range of practice materials and user populations.

The POWSM [12] model operates in a hybrid space between phonemic and phonetic representation, which can introduce occasional inaccuracies. The model's output may not always perfectly align with canonical IPA transcriptions, particularly for non-standard pronunciations or regional variations.

Nounce is designed exclusively for English to streamline initial development and ensure high-quality pronunciation assessment. Future development can expand support to additional languages, but this requires significant model adaptation, dictionary resources, and acoustic model training for each target language.

9.4.3 Future Directions

Future Directions for Turkish-Native Speakers: Fine-tuning the model to capture the nuances of people who use Turkish as their native language would improve accuracy for the target user population. This would involve collecting and curating training data from Turkish L1 speakers and adapting the model to better recognize common pronunciation patterns and errors specific to this population.

Language Expansion: Nounce is designed exclusively for English to streamline initial development and ensure high-quality pronunciation assessment. Future development can expand support to additional languages, but this requires significant model adaptation, dictionary resources, and acoustic model training for each target language.

Voice Cloning: Leverage voice cloning technology to generate target pronunciations with the user's voice for personalized feedback. This would allow learners to hear correct pronunciations in their own voice, potentially improving comprehension and retention.

LLM Analysis: Integrate a Large Language Model (LLM) to analyze performance data and provide user-specific, targeted feedback on each attempt, aggre-

gated over time. The LLM could identify patterns in errors, suggest personalized practice strategies, and provide contextual explanations for pronunciation challenges.

9.4.4 Ethical Considerations

Accent Variations & Linguistic Diversity: All dialects, accents, and language variants are linguistically valid. The system focuses on US English pronunciation as a widely recognized standard, but this choice does not minimize or devalue other accents. The primary goal is comprehensibility and effective communication rather than accent elimination. Users are encouraged to understand that accent reduction is optional and that clear communication is the objective.

User Autonomy & Privacy: Users control their learning process. Recordings are processed securely, and the platform prioritizes privacy in data handling and storage. Users must explicitly opt-in for any potential use of their data in model fine-tuning, voice cloning, or pronunciation generation features. Clear consent mechanisms and transparent data usage policies protect user privacy and maintain trust.

9.5 Conclusion

Nounce successfully demonstrates the practical application of state-of-the-art speech recognition models in an educational context. The system provides a functional, scalable platform for pronunciation assessment that combines technical excellence with pedagogical considerations. The project contributes to both the academic community through its reference implementation and to language learners through its accessible, detailed feedback system.

The development process highlighted the importance of iterative experimentation, careful architecture design, and user-centered development. The resulting system serves as a foundation for future enhancements and research in automated pronunciation assessment and language learning technology.

Bibliography

[1] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 33:12449–12460, 2020.

[2] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, et al. Wavlm: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.

[3] ELSA Corp. Elsa speak: Ai-powered english pronunciation coach. `https://elsaspeak.com/en/`, 2024. Accessed: November 2025.

[4] Evleksiz, Ümit Can and Bayram, Ömer Faruk. Application source code directory. `https://github.com/umitcan07/senior/tree/main/app`, 2024. Full-stack application codebase. Accessed: November 2025.

[5] Evleksiz, Ümit Can and Bayram, Ömer Faruk. Environment configuration. `https://github.com/umitcan07/senior/tree/main/app`, 2024. Environment variables and configuration files. Accessed: November 2025.

[6] Evleksiz, Ümit Can and Bayram, Ömer Faruk. Github actions ci/cd workflows. `https://github.com/umitcan07/senior/tree/main/.github/workflows`, 2024. Continuous integration and deployment pipelines. Accessed: November 2025.

[7] Evleksiz, Ümit Can and Bayram, Ömer Faruk. Project readme. `https://github.com/umitcan07/senior/blob/main/README.md`, 2024. Project documentation and setup instructions. Accessed: November 2025.

[8] Evleksiz, Ümit Can and Bayram, Ömer Faruk. Senior project repository. `https://github.com/umitcan07/senior/`, 2024. Boğaziçi University Computer Engineering Senior Project. Accessed: November 2025.

[9] Evleksiz, Ümit Can and Bayram, Ömer Faruk. Signal processing experiments directory. `https://github.com/umitcan07/senior/tree/main/sig/exp`, 2024. Jupyter notebooks for DSP and deep learning experiments. Accessed: November 2025.

[10] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *Proceedings of the International Conference on Machine Learning*, pages 369–376, 2006.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[12] Chin-Jou Li, Kalvin Chang, Shikhar Bharadwaj, Eunjung Yeo, Kwanghee Choi, Jian Zhu, David Mortensen, and Shinji Watanabe. Powsm: A phonetic open whisper-style speech foundation model. 2025.

[13] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *International Conference on Machine Learning (ICML)*, 2024.

[14] Michael McAuliffe, Michaela Socolof, Sarah Mihuc, Michael Wagner, and Morgan Sonderegger. Montreal forced aligner: Trainable text-speech alignment using kaldi. *Proceedings of Interspeech*, pages 498–502, 2017.

[15] Yifan Peng, Jinchuan Tian, Brian Yan, Dan Berrebbi, Xuankai Chang, Xinjian Li, Jiatong Shi, Siddhant Arora, William Chen, Roshan Sharma, et al. Owsm-ctc: An open encoder-only speech foundation model for speech recognition, translation, and language identification. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10189–10208, 2024.

[16] Pronounce Inc. Pronounce ai: Ai-powered speech feedback and pronunciation practice. `https://www.getpronounce.com/`, 2024. Accessed: November 2025.

[17] Silke M. Witt and Steve J. Young. Phone-level pronunciation scoring and assessment for interactive language learning. *Speech Communication*, 30(2-3):95–108, 2000.

[18] Junbo Zhang, Zhiwen Zhang, Yongqing Wang, Zhiyong Yan, Qiong Song, Yukai Huang, Ke Li, Daniel Povey, and Yujun Wang. speechocean762: An open-source non-native english speech corpus for pronunciation assessment. In *Proc. Interspeech*, pages 3710–3714, 2021.

[19] Guanlong Zhao, Sinem Sonsaat, Alif Silpachai, Ivana Lucic, Evgeny Chukharev-Hudilainen, John Levis, and Ricardo Gutierrez-Osuna. L2-arctic: A non-native english speech corpus. In *Proc. Interspeech*, pages 2783–2787, 2018.